Information Processing Letters 114 (2014) 634–638

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

Efficient algorithms for the longest common subsequence in k-length substrings

Sebastian Deorowicz [a], Szymon Grabowski [b,*]

[a] Institute of Informatics, Silesian University of Technology, Akademicka 16, 44-100 Gliwice, Poland
[b] Lodz University of Technology, Institute of Applied Computer Science, Al. Politechniki 11, 90-924 Łódź, Poland

ARTICLE INFO

Article history:
Received 18 November 2013
Received in revised form 13 May 2014
Accepted 17 May 2014
Available online 22 May 2014
Communicated by Ł. Kowalik

Keywords:
Algorithms
Combinatorial problems
LCSk
Sparse dynamic programming

ABSTRACT

Finding the longest common subsequence in k-length substrings (LCSk) is a recently proposed problem motivated by computational biology. This is a generalization of the well-known LCS problem in which matching symbols from two sequences A and B are replaced with matching non-overlapping substrings of length k from A and B. We propose several algorithms for LCSk, being non-trivial incarnations of the major concepts known from LCS research (dynamic programming, sparse dynamic programming, tabulation). Our algorithms make use of a linear-time and linear-space preprocessing finding the occurrences of all the substrings of length k from one sequence in the other sequence.

1. Introduction

In last years the famous longest common subsequence problem [4] gave rise to many related sequence similarity problems, often motivated by computational biology. One of them, proposed very recently by Benson et al. [3], is the *longest common subsequence in k-length substrings* problem, which can be defined as follows. Given two sequences, $A = a_1 a_2 \dots a_n$ and $B = b_1 b_2 \dots b_n$[1] over a common alphabet Σ, the task is to find the maximal ℓ such that there exist ℓ pairs of substrings of length k (called k-strings), $a_{i_e-k+1} \dots a_{i_e}$ and $b_{i_e-k+1} \dots b_{i_e}$, $1 \le e \le \ell$, where $a_{i_e-k+1} \dots a_{i_e}$ is equal to $b_{i_e-k+1} \dots b_{i_e}$ and $i_e + k \le i_{e+1}$ for any valid e (that is, the strings of length k taken from one of the sequences are non-overlapping). We will often use an alternative notation for a substring: instead of $a_i \dots a_j$ ($b_i \dots b_j$) we will write $A_{i\dots j}$ ($B_{i\dots j}$).

Benson et al. [3] presented a dynamic programming solution with $O(n^2)$ time complexity, assuming $k = O(1)$. In a general case the time becomes $O(kn^2)$, with $O(kn)$ space.

We first show how to obtain $O(n^2)$ time for unbounded k, and then present three more advanced algorithms. The first of them is based on the Hunt and Szymanski [7] approach (originally used for the LCS problem), applying the sparse dynamic programming paradigm. The second works better if the number of matches in the dynamic programming matrix is large and uses the observation that matches forming a longest common subsequence must be separated with gaps of size at least k. Its variant based on the van Emde Boas tree [11] is also briefly discussed. Finally, a tabulation-based algorithm is presented, with a logarithmic speedup over the quadratic-time dynamic programming algorithm. Our results are summarized in Table 1.

It is important to stress that our results are based on the ideas known from the LCS research, but the LCSk problem has its specific properties. Thus, a direct application of most of the classic techniques is not possible. In the following sections we point out the main obstacles.

* Corresponding author.
E-mail address: sgrabow@kis.p.lodz.pl (S. Grabowski).

[1] All the algorithms presented in this paper can easily be translated to the case of sequences of arbitrary lengths n and m, but we use the original problem definition.

http://dx.doi.org/10.1016/j.ipl.2014.05.009
0020-0190/

Table 1
Our results. The last column is the complexity of the extra space needed to extract a longest common subsequence. The extra time for this stage is not presented, but its complexity never exceeds the corresponding time complexity to find the subsequence length. Notation: r is the number of matches, $\ell \le n/k$ is the solution length.

Algorithm	Time complexity	Space complexity	Extraction space
DP (Section 2)	$O(n^2)$	$O(nk)$	$O(n^2)$
Sparse (Section 3)	$O(n + r \log \ell)$	$O(n + \min(r, n\ell))$	$O(r)$
Dense (Section 3)	$O(n^2/k + n(k \log n)^{2/3})$	$O(n)$	$O(n\ell)$
Dense-vEB (Section 3)	$O(n^2 \log\log n/k)$	$O(n \log\log n)$	$O(n\ell)$
DP-4R (Section 3)	$O(n^2/\log n)$	$O(n + nk/\log n)$	$O(n^2/\log n)$

2. The LCSk in $O(n^2)$ time

The cornerstone for any dynamic programming (DP) based solution for the LCSk problem will be the following recurrence. (It is closely related to the one given by Benson et al. We decided to introduce our own one, with match reporting at the end rather than start symbol of the k-string, since it simplifies the formulation of the algorithms in the rest of the paper.)

$$M(i,j) = \begin{cases} \max \begin{cases} M(i, j-1), \\ M(i-1, j), \end{cases} \\ \quad \text{if } A_{i-k+1\ldots i} \neq B_{j-k+1\ldots j}, \\ M(i-k, j-k) + 1, \\ \quad \text{if } A_{i-k+1\ldots i} = B_{j-k+1\ldots j}, \end{cases} \tag{1}$$

and the boundary conditions: $M(i, j) = 0$ for all valid i, j when $i < k$ or $j < k$. Any location (i, j) in M will be called a match if $A_{i-k+1\ldots i} = B_{j-k+1\ldots j}$.

Efficient computation of the recurrence (1) depends on quick tests if $A_{i-k+1\ldots i} = B_{j-k+1\ldots j}$. This can be achieved with the longest common extension (LCE) query, which can be performed in $O(1)$ time after $O(n)$-time preprocessing. This procedure builds an augmented suffix tree for solving the lowest common ancestor (LCA) queries in constant time [2], over the concatenated sequence $A\#B$, where # is a unique symbol (lexicographically largest) working as a terminator of A. Testing if $A_{i-k+1\ldots i} = B_{j-k+1\ldots j}$ translates to the question if $\mathrm{LCE}_{A\#B}(i-k+1, n+1+j-k+1) \ge k$.

We however propose an alternative $O(n)$-time preprocessing routine letting us access the successive matches to each k-string $A_{j-k+1\ldots j}$ in sequence B in constant time, and requiring $O(n)$ words of space. Since one list of matches is never longer than a row in the DP matrix, we can scan the list of matches when processing each row in a linear time, which results in overall $O(n^2)$ time for the matrix computation. The longest sequence itself may be extracted in a similar manner as in the DP algorithm for LCS, in $O(n)$ extra time and using $O(n^2)$ extra space.

Our preprocessing routine will also be used in the two algorithms described in Section 3 and the algorithm from Section 4. The procedure makes use of a suffix array for the concatenated sequence $B\#A$. We also build its longest common prefix (LCP) table; both operations can be accomplished in linear time and using linear space (see, e.g., [8,9]). The computed LCP values allow us to partition the sorted set of suffixes into maximal groups such that the LCP between successive items is at least k. In other words, suffixes from such groups have a prefix of length k symbols in common.

These k-string groups are radix-sorted according to the starting position of the suffix. To make it efficient ($O(n)$ time), the sort is performed once for all groups; all the suffixes are represented as pairs $(group\_id, start\_pos)$, where $group\_id$ is 1 for the first group, 2 for the second group, etc., in their position order. Let us denote the array with such pairs with S. After the sort, suffixes in the groups are kept together, in starting position order. Note that within a group all suffixes starting in B are located before any suffix starting in A.

We scan over all the items in S, except for the last one (which must correspond to the suffix starting with #), and we insert related data into another array, X of length $|A| + |B| = 2n$. More precisely, for each examined $S[i]$ we write in $X[S[i].start\_pos]$ a triple: (fa_pos, fb_pos, ng_pos), where fa_pos (fb_pos) is the position in S of the first suffix from this group starting in A (in B) and ng_pos is the position in S of the first suffix from the next group. This operation also takes linear time and the preprocessing is done.

As stated above, a rowwise scan requires fast access to all matches to successive $A_{i-k+1\ldots i}$ k-strings. It is now enough to examine the information stored at $X[S[i].start\_pos]$ (which in turn refers to S), to find the match locations in the row in $O(1)$ time per each.

Below we give two simple properties of the matrix M.

Lemma 1. *For each i and j the value $M(i, j)$ is the LCSk of the prefixes $A_{1\ldots i}$ and $B_{1\ldots j}$.*

The proof is rather straightforward and is very similar to the classic one for the LCS problem [4]. As a consequence, $M(n, n)$ is the solution of the LCSk problem. It is also easy to notice that $M(i, j+1) - M(i, j) \in \{0, 1\}$ and $M(i+1, j) - M(i, j) \in \{0, 1\}$ for all valid i and j. The following lemma describes a feature of the matrix M which will be crucial for both algorithms from Section 3.

Lemma 2. *Let vector $V(i)$, for any i, describe the changes in ith row of M, i.e., $V(i)$ stores the pairs $\langle M(i, j), j\rangle$ such that $M(i, j-1) + 1 = M(i, j)$. Then, each $\langle h, j\rangle \in V(i)$ implies that it is impossible that $\langle h+1, j'\rangle \in V(i')$, for any $i \le i' \le i+k$ and $j' < j+k$.*

Proof. Let us assume otherwise, so let $\langle h+1, j'\rangle \in V(i')$, for some $i \le i' \le i+k$ and $j' < j+k$. It means that $M(i', j') = h+1 = LCSk(A_{1\ldots i'}, B_{1\ldots j'})$. Truncating both sequences by k symbols does not change their LCSk or reduces it by 1, so $M(i'-k, j'-k) \ge h$. This is, however, impossible as $i'-k \le i$ and $j'-k < j$ and the leftmost

LCSk-*(A, B, k)

```
1   Compute MATCHLIST for A and B
2   THRESH[0] ← ⟨−∞, +∞⟩
3   for i ← 1 to k − 1 do
4       THRESH[i] ← copy(THRESH[i − 1])
5   for i ← k to n do
6       THRESH[i] ← copy(THRESH[i − 1])
7       σ_A ← get_k-string(A, i)
  {Sparse variant}
8       for each match x in MATCHLIST[σ_A] do
9           j′ ← THRESH[i − k].pred(x − k + 1)
10          h ← THRESH[i − k].rank(j′)
11          j″ ← THRESH[i].select(h + 1)
12          if x < j″ then
13              THRESH[i].decrease(j″, x)
14              if j″ = +∞ then
15                  THRESH[i].insert(+∞)
16  return |THRESH[n]| − 2
  {Dense variant}
8       for h ← 1 to |THRESH[i − 1]| − 1 do
9           j′ ← THRESH[i − k][h − 1]
10          x ← MATCHLIST[σ_A].succ(j′ + k − 1)
11          j″ ← THRESH[i][h]
12          if x < j″ then
13              THRESH[i][h] ← x
14              if j″ = +∞ then
15                  THRESH[i].insert(+∞)
16  return |THRESH[n]| − 2
```

Fig. 1. The sparse and dense DP algorithms for the LCSk problem.

cell of M among rows $0, \ldots, i$ containing value h is in column j. □

Simply speaking, the above lemma says that the increments in M are separated by at least k cells in both horizontal and vertical directions.

3. Two sparse dynamic programming algorithms

One of the major paradigms for solving LCS-related problems is *sparse dynamic programming* (SDP). The overall idea is to visit only those DP matrix cells which correspond to matches. As the number of matches, r, is usually significantly smaller than n^2, we can often expect a significant speedup over the standard DP procedure. The first such algorithm for the LCS problem was given by Hunt and Szymanski [7], with $O(r \log \ell)$ time in its basic variant, where $\ell \leq n$ is the LCS length. While this complexity is superquadratic in the worst case (i.e., for $r = \Theta(n^2)$), there exists a theoretical solution based on the HS approach which is free of this drawback [6, Section 5]. In this section we will present two SDP algorithms for the LCSk problem. The first of them applies the HS approach in an innovative way. It is important to notice that the properties of the LCSk problem make a direct application of the HS technique for LCS impossible (which will be explained later). This algorithm, called LCSk-Sparse, is presented in Fig. 1.

Assume that we are going to visit the matches rowwise, each row scanned from left to right. We start with a simple definition. $M(i, j)$ stores a match of *rank* h iff $A_{i-k+1\ldots i} = B_{j-k+1\ldots j}$ and $\mathrm{LCSk}(A_{1\ldots i}, B_{1\ldots j}) = h$. In the preprocessing (line 1), lists of successive occurrences of all k-strings from the sequence A are gathered (in $O(n)$ time), as described in Section 2. These lists will collectively be referred to as *MATCHLIST*, where *MATCHLIST*$[\sigma_A]$ stores the end positions of k-strings σ_A in sequence B, in ascending order.

In the standard LCS problem, any row depends only on the current and the previous row, so it is easy to implement the *THRESH* structure in the HS algorithm in various ways, e.g., as a simple array, binary search tree, van Emde Boas tree [11]. In each row the current version of *THRESH* is obtained via updates of *THRESH* obtained after processing the previous row. It is different in the LCSk problem, however, as we need to store k previous versions of *THRESH* (for k previous rows), and using naïve list copies adds an extra additive $n\ell$ term to the time complexity, which is prohibitive. To handle this issue, we propose to make use of a persistent data structure.

The main processing phase makes use of a persistent red-black tree [5] *THRESH* for maintaining the leftmost seen-so-far columns of matches of growing ranks. More precisely, accessing *THRESH*$[i][h]$ answers the question about the leftmost column in row i with a match of rank h. When processing row i, we will often be interested in accessing *THRESH*$[i-k]$, i.e., the state of this structure k rows earlier. For the first k rows of the DP matrix the structure *THRESH* contains only two sentinel values, $-\infty$ and $+\infty$ (lines 2–4), the former with rank 0. For each of the following rows, the state of *THRESH* from the previous row is modified only if the current match on *MATCHLIST*, of rank $h+1$, is in column x, and is less than the column j'' of the $(h+1)$-th value in *THRESH*$[i-1]$ (i.e., also *THRESH*$[i]$ so far). This modification (lines 12–13) involves decreasing a value in the structure, which may be implemented as one delete and one insert operation. Note that when the decreased value is the $+\infty$ sentinel, *THRESH*$[i]$ grows by one ($+\infty$ is again inserted at its end, in line 15). All operations on *THRESH* have logarithmic cost in the worst case, that is, $O(\log \ell)$, where $\ell \leq n/k$ is the LCSk length. The overall worst case time for the algorithm is thus $O(n + r \log \ell)$. The space consumption is usually determined by the number of matches r (we need $O(1)$ nodes of the persistent RB tree per each match).

The presented code only returns the length of an LCSk. Yet, to obtain the common subsequence itself we only have to modify the algorithm slightly, and with each entry in *THRESH*$[i]$ store a reference to the *THRESH* value used to compute the current value. This enables backtracking the solution in $O(\ell)$ extra time and using $O(r)$ words of space.

If the number of matches is close to n^2, a better solution is to use the algorithm LCSk-Dense (Fig. 1). A specific trait of this algorithm is skipping over some matches, when we are certain they cannot affect the algorithm's output. Making use of such a property is specific to the LCSk problem.

The first steps (lines 1–4) resemble the previous variant, but here the data structure *THRESH* is not persistent (and may be simply a dynamic array), hence the copy routine, used for each row i, has its cost linear in the size of *THRESH*$[i-1]$. The main loop, which is run for each row i, $i \geq k$, starts with making a copy of *THRESH*$[i-1]$ into *THRESH*$[i]$. Then, *THRESH*$[i]$ is traversed in order, and its

h-th value updated based on the current *MATCHLIST* and the *THRESH* in its state k rows earlier. More precisely, if the $(h-1)$-th value of $THRESH[i-k]$ is denoted with j' (line 9) and the nearest match on the current *MATCHLIST* in a column greater than or equal to $j'+k$, denoted by x (line 10), is less than the current h-th element of *THRESH* (line 11), then $THRESH[i][h]$ is updated to x (lines 12–13). As in the previous algorithm, $THRESH[i]$ may get longer by one (line 15). Finding the LCSk length needs access to k previous rows of *THRESH*, and as each of them contains at most $\ell+2 \le n/k+2$ items, the overall space is $O(n)$. Also in this algorithm the desired sequence may be backtracked in $O(\ell)$ extra time, if backlinks are stored together with *THRESH* entries. The memory use, however, is here $O(n\ell)$ words of space, due to physical copying of the $THRESH[i]$ structures.

Let us analyze the time complexity of this algorithm. It depends on how efficiently we can handle the successor queries in line 10. Binary search over $MATCHLIST[\sigma_A]$ gives a factor $\log n$. If k is small enough, however, we can remove the logarithmic factor. Two rows from the DP matrix will be considered equal (or one called a duplicate of another) if they have matches in the same set of columns. We start with a simple observation: for any $q \ge 1$ there cannot be more than q *distinct* rows with at least n/q matches in each of them. Let us use two thresholds, t_1 and t_2, where $1 \le t_2 < t_1 < n$. We set $t_1 = k \log n$ and let the rows with less than n/t_1 matches be called "sparse", those with at least n/t_1 and at most n/t_2 matches (the exact value of t_2 will be found later) be called "dense", and finally those rows with more than n/t_2 matches be called "superdense". In the sparse rows, we calculate the successor query in $O(\log n)$ time, spending $O(n^2 \log n/t_1) = O(n^2/k)$ time in total for them.

For the dense blocks, we partition each row into n/b blocks of size b cells each, where the exact value of b will be found later. Let us focus on a dense row i. For each block $M[i][j+1 \dots j+b]$ we first find $MATCHLIST[\sigma_A]$.succ$(j+1+k-1)$ and $MATCHLIST[\sigma_A]$.succ$(j+b+k-1)$ (using a linear scan over $MATCHLIST[\sigma_A]$) and if both values are the same, it means that all the cells in this block have the same successor used in line 10. If not, we associate with this block a list of all its b successors, one per each element from the block. These values are stored as dynamic arrays, one per block, of size 1 or b. The total time spent per a dense row is $O(n/b + nb/t_2)$. There are at most $t_1 = k \log n$ distinct dense rows, and finding the successors for all of them takes $O(k \log n(n/b + nb/t_2))$ time, minimized for $b = \sqrt{t_2}$ to $O(nk \log n/\sqrt{t_2})$ (duplicate rows obtain references to the already computed answers, in $O(n)$ total time).

Finally, superdense rows are processed naïvely in $O(n)$ time each, with $O(nt_2)$ total time. Overall, we obtain $O(n^2/k + nk \log n/\sqrt{t_2} + nt_2)$ time, which is minimized for $t_2 = (k \log n)^{2/3}$, to yield $O(n^2/k + n(k \log n)^{2/3})$ time. This reduces to simply $O(n^2/k)$ as long as $k = O((n/(\log n)^{2/3})^{3/5})$.

Alternatively, the successor queries may be handled with the famous van Emde Boas (vEB) tree [11], in $O(\log \log n)$ time. We need to maintain $O(n)$ such structures, using a variant with lazy initialization. In this way, the total time of the insertions (including initializations) is $O(n \log \log n)$ and so is the space consumption. The overall time complexity of this variant is $O(n^2 \log \log n/k)$ for any k.

As the HS approach was adopted for the LCSk problem in this section, it prompts the question if more sophisticated LCS algorithms along these lines, namely from Apostolico and Guerra [1] and Eppstein et al. [6], can be adopted as well. Both mentioned algorithms are based on the notion of *dominant matches*, which are a subset of all matches in the matrix. Roughly speaking, the lists of matches to visit change frequently in these algorithms, and the matches which are present in *THRESH* are removed from the match list. In the LCSk problem it seems that also matches in the successive k columns should be removed, but only for the next k rows after an update to *THRESH*. It seems difficult to handle these operations without compromising the overall time complexity and we were not able to devise an attractive algorithm of this kind.

4. Tabulation-based algorithm

The tabulation (also called "4-Russians") technique for dynamic programming algorithms consists in dividing the DP matrix into small blocks (usually $1 \times b$ or $b \times b$, for some b), such that the number of distinct blocks is small enough to be precomputed beforehand, e.g., with linear time-space resources. For the LCS problem this technique was first applied by Masek and Paterson [10].

Let us now present a tabulation-based algorithm for LCSk, called DP-4R; the reader needs to know the (purpose of the) data structures *THRESH* and *MATCHLIST* used in the previous section. In DP-4R, the current state of the list *THRESH* is represented as a bit-vector $THRESH_{\text{bin}}$ of length n and similarly the match lists, $MATCHLIST_{\text{bin}}$, for all k-strings from A are built (to avoid $O(n^2)$ bits of space in the worst case, these lists can be built on the fly, one per row). More precisely, $THRESH_{\text{bin}}[i][j] = 1$ iff $M(i,j) - M(i,j-1) = 1$, for any $1 \le j \le n$. Similarly, $MATCHLIST_{\text{bin}}[\sigma_A][j] = 1$ iff $B_{j-k+1\dots j} = \sigma_A$. For the current row i, $i \ge k$, each snippet $THRESH_{\text{bin}}[i][j+1 \dots j+b]$ depends only on:

- the snippet $THRESH_{\text{bin}}[i-1][j+1 \dots j+b]$,
- the snippet $THRESH_{\text{bin}}[i-k][j-k+1 \dots j-k+b]$,
- the snippet $MATCHLIST_{\text{bin}}[\sigma_A][j+1 \dots j+b]$,
- the difference $M(i,j) - M(i-1,j) \in \{0,1\}$,
- the difference $M(i,j) - M(i-k,j-k) \in \{0,1\}$.

(Both listed differences can be tracked easily during the rowwise snippet processing.)

Now, if $b = \Theta(\log n)$ with a small enough constant, we can compute the current snippet of $THRESH_{\text{bin}}[i]$ in constant time, with a lookup table built in the preprocessing (e.g., in $O(n)$ time), obtaining an $O(n^2/\log n)$-time algorithm. During the computations, the previous k rows of $THRESH_{\text{bin}}$ of length n bits need to be available, which makes the overall space use $O(n + nk/\log n)$ words.

It remains however to explain how $MATCHLIST_{\text{bin}}[\sigma_A] \times [j+1 \dots j+b]$ snippets are prepared. To this end, we note that all snippets from a row $MATCHLIST_{\text{bin}}[\sigma_A][1 \dots n]$ can

easily be created from a corresponding match list (found in the linear-time preprocessing) in $O(\max(n/\log n, r'))$ time, where $r' \leq n$ is the number of matches in this row. This means that all sparse rows, i.e. such for which $r' = O(n/\log n)$, pose no problem as the worst-case time of creating their $MATCHLIST_{\text{bin}}$ bit-vectors sums to $O(n^2/\log n)$. The number of *distinct* remaining (dense) rows in the matrix is however limited to less than $\log n$ (cf. a similar reasoning in Section 3 for the algorithm LCSk-Dense), hence the $O(n \log n + n^2/\log n)$ time for preparing their snippets, including their first occurrences and all duplicates, does not hamper the overall time complexity either.

An LCSk sequence can now be extracted basically like in the plain DP approach, in $O(n)$ time. To this end, the last 1 in $THRESH_{\text{bin}}[n]$ is found, with a linear scan from right to left, and its column j is the end position of the last k-string in the result. After that, we go to the row $n-k$ and column $j-k$, and scan left for the nearest 1, which will correspond to the penultimate k-string, and so on.

It is tempting to devise a similar algorithm based on bit logic rather than a precomputed table, but we suppose that obtaining $O(n^2/w)$ time, where $w \geq \log n$ is the machine word size, may be hard or even impossible for the LCSk problem.

5. Conclusions

We presented four algorithms, with respectively $O(n^2)$, $O(n + r \log \ell)$, $O(n^2/k + n(k \log n)^{2/3})$ and $O(n^2/\log n)$ time complexities, for the recently introduced problem of finding the longest common subsequence in k-length substrings. We used several major techniques known from the research on LCS and related problems: dynamic programming, sparse dynamic programming, tabulation. Their application to LCSk was, however, often non-trivial; for example using the Hunt–Szymanski approach required a persistent data structure to preserve an attractive time complexity. An interesting achievement is also the LCSk-Dense algorithm which provides a clearly subquadratic time complexity if k is large.

Acknowledgement

The work was supported by the Polish National Science Center upon decision DEC-2011/03/B/ST6/01588 (first author).

References

[1] A. Apostolico, C. Guerra, The longest common subsequence problem revisited, Algorithmica 2 (1–4) (1987) 315–336.
[2] M.A. Bender, M. Farach-Colton, The LCA problem revisited, in: G.H. Gonnet, D. Panario, A. Viola (Eds.), LATIN, in: Lect. Notes Comput. Sci., vol. 1776, Springer, 2000, pp. 88–94.
[3] G. Benson, A. Levy, B.R. Shalom, Longest common subsequence in k length substrings, in: N.R. Brisaboa, O. Pedreira, P. Zezula (Eds.), SISAP, in: Lect. Notes Comput. Sci., vol. 8199, Springer, 2013, pp. 257–265.
[4] M. Crochemore, C. Hancart, T. Lecroq, Algorithms on Strings, Cambridge University Press, New York, USA, 2007.
[5] J.R. Driscoll, N. Sarnak, D.D. Sleator, R.E. Tarjan, Making data structures persistent, J. Comput. Syst. Sci. 38 (1) (1989) 86–124.
[6] D. Eppstein, Z. Galil, R. Giancarlo, G.F. Italiano, Sparse dynamic programming I: linear cost functions, J. ACM 39 (3) (1992) 519–545.
[7] J.W. Hunt, T.G. Szymanski, A fast algorithm for computing longest common subsequences, Commun. ACM 20 (5) (1977) 350–353.
[8] J. Kärkkäinen, P. Sanders, S. Burkhardt, Linear work suffix array construction, J. ACM 53 (6) (2006) 918–936.
[9] T. Kasai, G. Lee, H. Arimura, S. Arikawa, K. Park, Linear-time longest-common-prefix computation in suffix arrays and its applications, in: A. Amir, G.M. Landau (Eds.), CPM, in: Lect. Notes Comput. Sci., vol. 2089, Springer, 2001, pp. 181–192.
[10] W. Masek, M. Paterson, A faster algorithm computing string edit distances, J. Comput. Syst. Sci. 20 (1) (1980) 18–31.
[11] P. van Emde Boas, Preserving order in a forest in less than logarithmic time and linear space, Inf. Process. Lett. 6 (3) (1977) 80–82.